IMPLEMENTACION DE LA ESTRUCTURA COMPLETA DE UNA BASE DE DATOS.

PRESENTADO POR:

-NATHALY RIVAS ARISTIZABAL.

CURSO:
BASES DE DATOS.

TUTORA:
ORALIA CORTÉS CARVAJAL.

G100

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO.
2026

1. IDENTIFICACIÓN Y JUSTIFICACIÓN DEL SGBD SELECCIONADO

Para la implementación del modelo físico de mi proyecto Red Social Estudiantil Pascualina, seleccioné el Sistema Gestor de Bases de Datos MySQL (versión 8.0), apoyándome en la herramienta de administración MySQL Workbench.

*ANÁLISIS COMPARATIVO Y JUSTIFICACIÓN.

SGBD Evaluado	Tipo / Arquitectura	Ventajas principales	Desventajas / Limitaciones	Mi elección
MySQL 8.0	Relacional (RDBMS)	Alto rendimiento en consultas de lectura (SELECT). Excelente integración gráfica con MySQL Workbench. Estándar amplio en la industria y despliegue rápido. Soporte robusto de claves foráneas con motor InnoDB.	Menor soporte nativo de tipos de datos avanzados complejos comparado con PostgreSQL.	Elegí este SGBD: Consideré que es el sistema ideal para una red social donde prevalecen las lecturas rápidas de muros, perfiles y listados de eventos.
PostgreSQL	Objeto-Relacional	Altamente estricto en tipos de datos y estándares SQL. Potente motor para consultas avanzadas e indexación compleja.	Mayor consumo de recursos de memoria en mi servidor local de prueba.	Descarté: Evalué que no requería la complejidad objeto-relacional para el modelo físico actual de mi tarea.
MongoDB	NoSQL (Documentos)	Esquema flexible sin tablas fijas.	Carece de integridad referencial estricta nativa (FOREING	Descarté: La guía exigía un modelo relacional estricto con

		Escalabilidad horizontal nativa.	KEYS y cascadas)	restricciones DDL de integridad.

Justificación final: Elegí MySQL con el motor de almacenamiento InnoDB porque me garantiza el cumplimiento de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). De esta forma, me aseguré de que cada operación en la red social como registrar un estudiante o unirse a un grupo mantenga la integridad mediante restricciones de clave primaria y foránea en tiempo real.

2. TRANSFORMACIÓN DEL MODELO LÓGICO AL MODELO FÍSICO.

Para realizar la transición desde mi modelo conceptual y lógico de la Fase 2 hacia el modelo físico en MySQL, apliqué las siguientes reglas de transformación:

1. Entidades a Tablas: Traduje las 7 entidades de mi modelo lógico (`Estudiante`, `Conexión`, `Publicación`, `Grupo`, `Miembro_Grupo`, `Evento`, `Asistente_Evento`) en tablas físicas con nombres en minúsculas y plurales descriptivos.
2. Normalización (3FN): Me aseguré de que cada tabla cumpliera con la Tercera Forma Normal (3FN). Eliminé atributos multivaluados los separé en tablas asociativas y dependencias transitivas.
3. Claves primarias (PK): Definí llaves primarias con enteros autoincrementables (`INT AUTO_INCREMENT PRIMARY KEY`) para las entidades simples, y diseñé claves compuestas (`PK1, PK2`) para las tablas asociativas relacionales.
4. Mapeo de atributos: Mapeé los atributos lógicos a tipos de datos físicos nativos de MySQL (`VARCHAR`, `INT`, `TEXT`, `DATETIME`, `BOOLEAN`).

3. DICCIONARIO DE DATOS COMPLETO

A continuación describo la estructura física que diseñe e implementé para cada una de las 7 tablas de mi script DDL:

3.1. Tabla: `estudiantes` (Tabla Padre Central)

Almacena la información académica y personal de los alumnos registrados.

Nombre del campo	Tipo de dato y longitud	Restricciones / Claves	Justificacion tecnica
estudiante_id	INT	PK, AUTO_INCREMENT	Elegí `INT` en lugar de `BIGINT` porque un entero

			de 4 bytes me permite gestionar hasta 2,147,483,647 registros, capacidad suficiente para la universidad, optimizando el tamaño del índice en memoria.
nombre	VARCHAR (100)	NOT NULL	Permite guardar nombres y apellidos completos. Preferí `VARCHAR` sobre `TEXT` porque limita el tamaño máximo evitando desperdicio de disco y optimiza las búsquedas.
correo	VARCHAR (150)	NOT NULL, UNIQUE	Pensé esta restricción única para garantizar que no existan cuentas duplicadas con el mismo correo institucional.
carrera	VARCHAR (100)	NOT NULL	Asigné esta longitud para albergar nombres formales de programas académicos como Ingeniería de Software.
semestre	INT	CHECK (semestre > 0 AND semestre <= 12)	Incluí una restricción `CHECK` para validar que el nivel académico ingresado esté dentro del rango institucional real.

intereses	TEXT	Opcional (NULL)	Usé TEXT porque la lista de gustos del alumno puede ser de longitud variable o extensa.
habilidades	TEXT	Opcional (NULL)	Elegí este tipo para permitir redactar múltiples destrezas sin una restricción corta de caracteres.
fecha_registro	DATETIME	DEFAULT CURRENT_TIMESTAMP	Configuré este campo para registrar la fecha y hora exacta de creación del perfil automáticamente.

3.2. Tabla: `conexiones` (Tabla Asociativa Autorreferencial)

Modela los enlaces de seguimiento y mentoría académica entre estudiantes.

Nombre del campo	Tipo de dato y longitud	Restricciones / Claves	Justificacion tecnica
seguidor_id	INT	PK, FK	Identificador del estudiante que realiza el seguimiento. Apunta a `estudiantes.estudiante_id`
seguido_id	INT	PK, FK	Identificador del estudiante seguido. Apunta a `estudiantes.estudiante_id`.
es_mentor	BOOLEAN	DEFAULT FALSE	Decidí usar un valor lógico (0 o 1) para clasificar si la

			relación es de tutoría o simple seguimiento casual.
fecha_conexion	DATETIME	DEFAULT CURRENT_TIMESTAMP	Registra el momento en que se estableció el vínculo.

Clave primaria compuesta: `PRIMARY KEY (seguidor_id, seguido_id)`. La configuré para prevenir que un estudiante siga dos veces al mismo compañero.

3.3. Tabla: `publicaciones` (Tabla Hija)

Registra los mensajes, dudas, recursos o actualizaciones que los estudiantes comparten en el muro.

Nombre del campo	Tipo de dato y longitud	Restricciones / Claves	Justificacion tecnica
publicacion_id	INT	PK, `AUTO_INCREMENT`	Identificador único de la publicación.
estudiante_id	INT	FK, `NOT NULL`	Clave foránea que vincula la publicación con su autor (`estudiantes.estudiante_id`).
contenido	TEXT	NOT NULL	Elegí `TEXT` (hasta 65,535 caracteres) para dar libertad al alumno de escribir posts extensos, guías o código sin truncar la información.
categoria	VARCHAR(50)	DEFAULT 'General'	Pensé esta columna para clasificar los mensajes (ej. *Tareas*, *Memes*,

			Avisos).
fecha_publicacion	DATETIME	DEFAULT CURRENT_TIMESTAMP	Permite ordenar las publicaciones del muro de forma cronológica.

3.4. Tabla: `grupos` (Tabla Padre/Hija)

Modela los grupos de estudio, comunidades o clubes de la universidad.

Nombre del campo	Tipo de dato y longitud	Restricciones / Claves	Justificacion tecnica
grupo_id	INT	PK, AUTO_INCREMENT	Identificador único de cada comunidad.
nombre	VARCHAR(120)	NOT NULL	Título del grupo de estudio o club.
descripcion	TEXT	Opcional (NULL)	Incluí esta opción para explicar el objetivo del colectivo o sus reglas.
creador_id	INT	FK, NOT NULL	Identificador del estudiante creador (`estudiantes.estudiante_id`).
fecha_creacion	DATETIME	DEFAULT CURRENT_TIMESTAMP	Marca de tiempo de fundación del grupo.

3.5. Tabla: `miembros_grupo` (Tabla Asociativa N:M)

Registra qué estudiantes pertenecen a cuáles grupos y con qué rol.

Nombre del campo	Tipo de dato y longitud	Restricciones / Claves	Justificacion tecnica

grupo_id	INT	PK, FK	Clave foránea referenciando a `grupos.grupo_id`.
estudiante_id	INT	PK, FK	Clave foránea referenciando a `estudiantes.estudiante_id`.
rol	VARCHAR(50)	DEFAULT 'Miembro'	Definí este campo para establecer jerarquías dentro del grupo como administrador o miembro.
fecha_union	DATETIME	DEFAULT 'Miembro'	Registra el ingreso del alumno a la comunidad.

Clave primaria compuesta: `PRIMARY KEY (grupo_id, estudiante_id)`. La implementé para evitar registros duplicados de un mismo estudiante en un mismo grupo.

3.6. Tabla: `eventos` (Tabla Padre/Hija)

Almacena reuniones de estudio, talleres, hackatones o eventos sociales.

Nombre del campo	Tipo de dato y longitud	Restricciones / Claves	Justificacion tecnica
evento_id	INT	PK, `AUTO_INCREMENT`	Identificador único de la actividad.
titulo	VARCHAR(150)	NOT NULL	Nombre descriptivo de la convocatoria.
descripcion	TEXT	Opcional (`NULL`)	Detalle de la agenda o temas a tratar.
fecha_evento	DATETIME	NOT NULL	Fecha y hora exacta de realización del evento.

ubicacion	VARCHAR(150)	Opcional (NULL)	Consideré esta longitud adecuada para direcciones físicas como bloque 12- 303 o enlaces a salas virtuales.
organizador_id	INT	FK, NOT NULL	Clave foránea que referencia al estudiante organizador.
grupo_id	INT	FK, Opcional (NULL)	Clave foránea opcional que vincula el evento a un grupo si fue creado por este

3.7. Tabla: `asistentes_evento` (Tabla Asociativa N:M)

Gestión del registro y confirmación de asistencia a eventos.

Nombre del campo	Tipo de dato y longitud	Restricciones / Claves	Justificacion tecnica
evento_id	INT	PK, FK	Clave foránea que referencia a `eventos.evento_id`
estudiante_id	INT	PK, FK	Clave foránea que referencia a `estudiantes.estudiante_id`.
estado_confirmacion	VARCHAR(30)	DEFAULT 'Confirmado'	Asigné este campo para rastrear la confirmación (ej. Confirmado, En espera, Cancelado).

Clave primaria compuesta: `PRIMARY KEY (evento_id, estudiante_id)`. La diseñé para impedir la doble confirmación de un estudiante al mismo evento.

4. MATRIZ DE RELACIONES E INTEGRIDAD REFERENCIAL QUE ESTABLECÍ.

Para asegurarme de que la base de datos no conserve registros huérfanos y mantenga la coherencia en todo momento, establecí cláusulas de acción referencial `ON DELETE` y `ON UPDATE`:

Tabla padre	Tabla hija	Campos involucrados FK	Acciones referenciales	Justificacion de la regla
estudiantes	conexiones	`seguidor_id`, `seguido_id` → `estudiante_id`	`ON DELETE CASCADE` `ON UPDATE CASCADE`	Decidí que si un estudiante elimina su cuenta, se borren automáticamente todas sus conexiones de seguimiento para evitar enlaces rotos.
estudiantes	publicaciones	`estudiante_id` → `estudiante_id`	`ON DELETE CASCADE` `ON UPDATE CASCADE`	Configuré que al dar de baja a un usuario, sus publicaciones se eliminen en cascada liberando espacio.
estudiantes	grupos	`creador_id` → `estudiante_id`	`ON DELETE RESTRICT` `ON UPDATE CASCADE`	Establecí que se impida eliminar la cuenta de un estudiante si es el único creador de un grupo activo, obligando a reasignar el grupo antes.
grupos	miembros_grupo	`grupo_id` → `grupo_id`	`ON DELETE`	Decidí que si se elimina un grupo, la lista

			CASCADE ON UPDATE CASCADE	de miembros inscritos se limpie automáticamente.
estudiantes	miembros_grupo	estudiante_id → estudiante_id	ON DELETE CASCADE ON UPDATE CASCADE	Definí que si un alumno se retira, su membresía en los grupos se elimine en cascada.
estudiantes	eventos	organizador_id → estudiante_id	ON DELETE CASCADE ON UPDATE CASCADE	Pensé que si se elimina la cuenta del organizador, los eventos convocados por él se remuevan del sistema.
grupos	eventos	grupo_id → grupo_id	ON DELETE SET NULL ON UPDATE CASCADE	Elegí SET NULL porque si un grupo se borra, los eventos no deben eliminarse; quedan vigentes pero el campo grupo_id pasa a ser nulo.
eventos	asistentes_evento	evento_id → evento_id	ON DELETE CASCADE ON UPDATE CASCADE	Configuré que si se cancela un evento, la lista de asistencia se elimine automáticamente.

5. REFERENCIA AL SCRIPT DDL Y ESTRUCTURA DE EJECUCIÓN

El script SQL completo lo estructuré y guardé en mi repositorio bajo la ruta estándar:

`Tarea3/script.sql`

Mi lógica y orden de ejecución del script:

1. Creación de entorno: Comencé creando la base de datos red_social mediante CREATE DATABASE IF NOT EXISTS red_social; y luego la seleccioné con USE red_social;.
2. Eliminación en orden inverso: Incluí sentencias DROP TABLE IF EXISTS ordenadas desde las tablas hijas/asociativas hacia las tablas padre (asistentes_evento → eventos → miembros_grupo → grupos → publicaciones → conexiones → estudiantes). Hice esto para evitar errores de violación de integridad referencial durante instalaciones previas.
3. Construcción gradual: Escribí primero la tabla central estudiantes, luego las tablas padre independientes y finalicé con las asociativas de llaves compuestas.

6. Mis conclusiones individuales y reflexión de aprendizaje

1. Transformación del modelo lógico a físico: El desarrollo de esta tarea me permitió comprender la importancia de llevar mis diseños teóricos a un motor real de bases de datos como MySQL. Aprendí que cada decisión sobre los tipos de datos (por ejemplo, elegir `VARCHAR` sobre `TEXT` o `INT` sobre `BIGINT`) impacta directamente en el rendimiento del sistema y en la optimización de memoria.
2. Dominio de la integridad referencial: La definición manual de restricciones como `ON DELETE CASCADE` y `ON DELETE SET NULL` me aclaró cómo las reglas de negocio de una plataforma digital se protegen desde la arquitectura misma de la base de datos, evitando inconsistencias o datos aislados.
3. Retos y soluciones durante mi práctica: Enfrentar la ejecución tanto en la consola de comandos de MySQL como en la interfaz gráfica de MySQL Workbench me dio mucha flexibilidad técnica. Superé las dificultades iniciales de configuración del servidor local en Windows y aprendí a diagnosticar mensajes de error de sintaxis DDL, lo que fortaleció mis competencias para el desarrollo de software.